

<https://gemini.google.com/app/5a60b2005f74036c>

Response: Which target programming language would you like to use for the **Cooking Assistance** domain model?

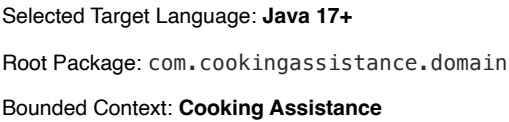
1. **Java** (Recommended)
2. **Kotlin**
3. **C#**
4. **TypeScript**
5. **Other** (e.g., Python, Go, Rust)

User prompt: Java









1. External Reference IDs (com.cookingassistance.domain.external)

<https://gemini.google.com/gem/7a2363af6bef/5a60b2005f74036c>

```

import java.util.Objects;
import java.util.UUID;

public record HowToStepId(UUID value) {
    public HowToStepId {
        Objects.requireNonNull(value, "HowToStepId cannot be null");
    }
}

package com.cookingassistance.domain.external;

import java.util.Objects;
import java.util.UUID;

public record IngredientId(UUID value) {
    public IngredientId {
        Objects.requireNonNull(value, "IngredientId cannot be null");
    }
}

```

2. HelpRequest Aggregate (com.cookingassistance.domain.helprequest)

```

package com.cookingassistance.domain.helprequest;

import java.util.Objects;
import java.util.UUID;

public record HelpRequestId(UUID value) {
    public HelpRequestId {
        Objects.requireNonNull(value, "HelpRequestId cannot be null");
    }
}

package com.cookingassistance.domain.helprequest;

public enum HelpRequestType {
    INGREDIENT_SUBSTITUTE,
    PREPARATION_STEP_EXPLANATION,
    STEPS_TO_MITIGATE_CATASTROPHE,
    MENU_PROPOSAL
}

package com.cookingassistance.domain.helprequest;

public enum ProviderType {
    CHEF,
    GRANDMA_AVATAR,
    COMMUNITY
}

package com.cookingassistance.domain.helprequest;

public enum HelpRequestStatus {
    OPEN,
    ANSWERED,
    CLOSED
}

package com.cookingassistance.domain.helprequest;

import com.cookingassistance.domain.external.*;
import com.cookingassistance.domain.exceptions.DomainRuleViolationException;

import java.net.URI;
import java.util.*;

public final class HelpRequest {
    private final HelpRequestId id;
    private final UserId requester;
    private final String title;
    private final HelpRequestType type;
    private final String description;
    private final RecipeId recipe;
    private final HowToStepId howToStep;
    private final List<IngredientId> ingredients;
    private final Set<ProviderType> preferredProviders;
    private HelpRequestStatus status;
    private final List<URI> pictures;

    public HelpRequest(
        HelpRequestId id,

```

```

        UserId requester,
        String title,
        HelpRequestType type,
        String description,
        RecipeId recipe,
        HowToStepId howToStep,
        List<IngredientId> ingredients,
        Set<ProviderType> preferredProviders,
        List<URI> pictures
    ) {
        this.id = Objects.requireNonNull(id, "HelpRequestId required");
        this.requester = Objects.requireNonNull(requester, "Requester required");
        this.title = Objects.requireNonNull(title, "Title required");
        this.type = Objects.requireNonNull(type, "HelpRequestType required");
        this.description = Objects.requireNonNull(description, "Description required");

        // Rule: Recipe mandatory for PREPARATION_STEP_EXPLANATION and INGREDIENT_SUBSTITUTE
        if ((type == HelpRequestType.PREPARATION_STEP_EXPLANATION || type == HelpRequestType.INGREDIENT_SU
            throw new DomainRuleViolationException("Recipe is mandatory for type " + type);
        }
        this.recipe = recipe;

        // Rule: howToStep only for PREPARATION_STEP_EXPLANATION
        if (howToStep != null && type != HelpRequestType.PREPARATION_STEP_EXPLANATION) {
            throw new DomainRuleViolationException("howToStep is only allowed for PREPARATION_STEP_EXPLANA
        }
        this.howToStep = howToStep;

        // Rule: ingredients only for INGREDIENT_SUBSTITUTE
        List<IngredientId> ingList = ingredients == null ? List.of() : List.copyOf(ingredients);
        if (!ingList.isEmpty() && type != HelpRequestType.INGREDIENT_SUBSTITUTE) {
            throw new DomainRuleViolationException("Ingredients are only allowed for INGREDIENT_SUBSTITUTE
        }
        this.ingredients = ingList;

        // Preferred providers validation: 1..2 items
        Objects.requireNonNull(preferredProviders, "Preferred providers required");
        if (preferredProviders.isEmpty() || preferredProviders.size() > 2) {
            throw new DomainRuleViolationException("Must specify 1 to 2 preferred providers");
        }

        // Rule: CHEF is exclusive (size = 1) and only for MENU_PROPOSAL
        if (preferredProviders.contains(ProviderType.CHEF)) {
            if (preferredProviders.size() > 1) {
                throw new DomainRuleViolationException("Chef provider must be requested exclusively");
            }
            if (type != HelpRequestType.MENU_PROPOSAL) {
                throw new DomainRuleViolationException("Chef provider is only allowed for MENU_PROPOSAL");
            }
        }
        this.preferredProviders = Set.copyOf(preferredProviders);

        this.pictures = pictures == null ? List.of() : List.copyOf(pictures);
        this.status = HelpRequestStatus.OPEN;
    }

    public void markAnswered() {
        if (this.status == HelpRequestStatus.CLOSED) {
            throw new DomainRuleViolationException("Cannot answer a closed HelpRequest");
        }
        this.status = HelpRequestStatus.ANSWERED;
    }

    public void close(boolean hasAcceptedHelp) {
        // Rule: At least one Help must be accepted to close
        if (!hasAcceptedHelp) {
            throw new DomainRuleViolationException("At least one Help must be accepted to close the reques
        }
        this.status = HelpRequestStatus.CLOSED;
    }

    public HelpRequestId getId() { return id; }
    public UserId getRequester() { return requester; }
    public String getTitle() { return title; }
    public HelpRequestType getType() { return type; }
    public String getDescription() { return description; }
    public Optional<RecipeId> getRecipe() { return Optional.ofNullable(recipe); }
    public Optional<HowToStepId> getHowToStep() { return Optional.ofNullable(howToStep); }
    public List<IngredientId> getIngredients() { return ingredients; }
    public Set<ProviderType> getPreferredProviders() { return preferredProviders; }

```

```

    public HelpRequestStatus getStatus() { return status; }
    public List<URI> getPictures() { return pictures; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        HelpRequest that = (HelpRequest) o;
        return Objects.equals(id, that.id);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}

package com.cookingassistance.domain.helprequest;

import java.util.Optional;

public interface HelpRequestRepository {
    void save(HelpRequest helpRequest);
    Optional<HelpRequest> findById(HelpRequestId id);
}

```

3. Help Aggregate (com.cookingassistance.domain.help)

```

package com.cookingassistance.domain.help;

import java.util.Objects;
import java.util.UUID;

public record HelpId(UUID value) {
    public HelpId {
        Objects.requireNonNull(value, "HelpId cannot be null");
    }
}

package com.cookingassistance.domain.help;

public enum Unit {
    PIECE, CUP, TABLE_SPOON, TEA_SPOON, FLUID_OUNCE, PINT, QUART, POUND, OUNCE, GRAM, LITTER, MILLILITER,
}

package com.cookingassistance.domain.help;

public enum Meal {
    BREAKFAST, LUNCH, DINNER, SUPPER
}

package com.cookingassistance.domain.help;

import java.util.Objects;

public record Ingredient(String name, double value, Unit unit) {
    public Ingredient {
        Objects.requireNonNull(name, "Name required");
        Objects.requireNonNull(unit, "Unit required");
        if (value <= 0) throw new IllegalArgumentException("Value must be positive");
    }
}

package com.cookingassistance.domain.help;

import com.cookingassistance.domain.external.RecipeId;
import java.util.Objects;

public record Course(int dish, RecipeId recipe) {
    public Course {
        Objects.requireNonNull(recipe, "Recipe required");
    }
}

package com.cookingassistance.domain.help;

import com.cookingassistance.domain.helprequest.HelpRequestType;

public sealed interface Answer permits
    IngredientSubstituteAnswer,
    PreparationStepExplanationAnswer,

```

```

        StepsToMitigateCatastropheAnswer,
        MenuProposalAnswer {
    HelpRequestType getRequestType();
}

package com.cookingassistance.domain.help;

import com.cookingassistance.domain.external.RecipeId;
import com.cookingassistance.domain.helprequest.HelpRequestType;
import java.util.List;
import java.util.Objects;

public record IngredientSubstituteAnswer(
    RecipeId recipe,
    List<SolutionItem> solutions
) implements Answer {
    public record SolutionItem(Ingredient ingredient, Ingredient replacesIngredient) {}

    public IngredientSubstituteAnswer {
        Objects.requireNonNull(recipe, "Recipe required");
        Objects.requireNonNull(solutions, "Solutions required");
        if (solutions.isEmpty()) {
            throw new IllegalArgumentException("At least one solution required");
        }
    }

    @Override
    public HelpRequestType getRequestType() {
        return HelpRequestType.INGREDIENT_SUBSTITUTE;
    }
}

package com.cookingassistance.domain.help;

import com.cookingassistance.domain.external.HowToStepId;
import com.cookingassistance.domain.external.RecipeId;
import com.cookingassistance.domain.helprequest.HelpRequestType;
import java.net.URI;
import java.util.List;
import java.util.Objects;

public record PreparationStepExplanationAnswer(
    RecipeId recipe,
    HowToStepId howToStep,
    String solution,
    List<URI> images
) implements Answer {
    public PreparationStepExplanationAnswer {
        Objects.requireNonNull(howToStep, "HowToStep required");
        Objects.requireNonNull(solution, "Solution text required");
        images = images == null ? List.of() : List.copyOf(images);
    }

    @Override
    public HelpRequestType getRequestType() {
        return HelpRequestType.PREPARATION_STEP_EXPLANATION;
    }
}

package com.cookingassistance.domain.help;

import com.cookingassistance.domain.external.RecipeId;
import com.cookingassistance.domain.helprequest.HelpRequestType;
import java.util.List;
import java.util.Objects;

public record StepsToMitigateCatastropheAnswer(
    RecipeId recipe,
    List<String> solutions
) implements Answer {
    public StepsToMitigateCatastropheAnswer {
        Objects.requireNonNull(solutions, "Solutions required");
        if (solutions.isEmpty()) {
            throw new IllegalArgumentException("At least one solution required");
        }
    }

    @Override
    public HelpRequestType getRequestType() {
        return HelpRequestType.STEPS_TO_MITIGATE_CATASTROPHE;
    }
}

```

```

    }
}

package com.cookingassistance.domain.help;

import com.cookingassistance.domain.exceptions.DomainRuleViolationException;
import com.cookingassistance.domain.helprequest.HelpRequestType;
import java.util.List;
import java.util.Objects;

public record MenuProposalAnswer(
    String note,
    int scaling,
    Meal meal,
    String howToServe,
    List<Course> courses
) implements Answer {
    public MenuProposalAnswer {
        Objects.requireNonNull(note, "Note required");
        Objects.requireNonNull(meal, "Meal required");
        Objects.requireNonNull(courses, "Courses required");

        // Rule: 1..10 courses
        if (courses.isEmpty() || courses.size() > 10) {
            throw new DomainRuleViolationException("Menu proposal must contain between 1 and 10 courses");
        }
    }

    @Override
    public HelpRequestType getRequestType() {
        return HelpRequestType.MENU_PROPOSAL;
    }
}

package com.cookingassistance.domain.help;

import com.cookingassistance.domain.external.UserId;
import com.cookingassistance.domain.helprequest.HelpRequestId;
import com.cookingassistance.domain.helprequest.HelpRequestType;
import com.cookingassistance.domain.helprequest.ProviderType;
import com.cookingassistance.domain.exceptions.DomainRuleViolationException;

import java.util.Objects;

public final class Help {
    private final HelpId id;
    private final HelpRequestId helpRequest;
    private final UserId helpProviderUser;
    private final String universalTitle;
    private final ProviderType helpProviderType;
    private final Answer answer;
    private boolean accepted;

    public Help(
        HelpId id,
        HelpRequestId helpRequest,
        UserId helpProviderUser,
        String universalTitle,
        ProviderType helpProviderType,
        HelpRequestType targetRequestType,
        Answer answer
    ) {
        this.id = Objects.requireNonNull(id, "HelpId required");
        this.helpRequest = Objects.requireNonNull(helpRequest, "HelpRequestId required");
        this.helpProviderUser = Objects.requireNonNull(helpProviderUser, "HelpProvider User required");
        this.universalTitle = Objects.requireNonNull(universalTitle, "Universal title required");
        this.helpProviderType = Objects.requireNonNull(helpProviderType, "ProviderType required");
        this.answer = Objects.requireNonNull(answer, "Answer required");

        // Rule: Answer type must match request type
        if (answer.getRequestType() != targetRequestType) {
            throw new DomainRuleViolationException("Answer payload type must match the target HelpRequest");
        }
        this.accepted = false;
    }

    public void accept() {
        this.accepted = true;
    }
}

```

```

    public HelpId getId() { return id; }
    public HelpRequestId getHelpRequestId() { return helpRequestId; }
    public UserId getHelpProviderUser() { return helpProviderUser; }
    public String getUniversalTitle() { return universalTitle; }
    public ProviderType getHelpProviderType() { return helpProviderType; }
    public Answer getAnswer() { return answer; }
    public boolean isAccepted() { return accepted; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Help help = (Help) o;
        return Objects.equals(id, help.id);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}

package com.cookingassistance.domain.help;

import java.util.List;
import java.util.Optional;

public interface HelpRepository {
    void save(Help help);
    Optional<Help> findById(HelpId id);
    List<Help> findByHelpRequestId(com.cookingassistance.domain.helprequest.HelpRequestId helpRequestId);
}

package com.cookingassistance.domain.exceptions;

public class DomainRuleViolationException extends RuntimeException {
    public DomainRuleViolationException(String message) {
        super(message);
    }
}

```

Model Decision Record (MODEL-DECISIONS.md)

Model Decision Record: Cooking Assistance Domain Model

```

- **Bounded Context**: Cooking Assistance[cite: 1, 2]
- **Language**: Java 17+[cite: 1, 2]
- **Root Package**: `com.cookingassistance.domain`[cite: 1, 2]
- **Terms in Scope**: `HelpRequest`, `Help`, `Answer`, `ProviderType`, `HelpRequestType`, `HelpRequestStat`
- **Terms Excluded / Externalized**: `Recipe`, `HowToStep`, `Ingredient` (catalog entity), `User` (held st

```

1. Classification Table

Term	Kind	Evidence from Glossary	Confidence
`HelpRequest`	Aggregate Root	Has `help requestId` ID, root of request workflow, referenced by `Help`	
`HelpRequestId`	Value Object	Identifier for `HelpRequest` aggregate[cite: 2].	High
`Help`	Aggregate Root	Has `help id` ID, independent lifecycle (`accepted`)[cite: 1].	High
`HelpId`	Value Object	Identifier for `Help` aggregate[cite: 1].	High
`HelpRequestType`	Enum Value Object	Closed taxonomy: `Ingredient Substitute`, `Preparation Step Expl`	
`ProviderType`	Enum Value Object	Closed set: `Chef`, `Grandma Avatar`, `Community`[cite: 1, 2].	Hi
`HelpRequestStatus`	Enum Value Object	Status progression: `Open`, `Answered`, `Closed`[cite: 2].	H
`Answer`	Sealed Interface V0	Polymorphic payload discriminated by `request type`[cite: 1].	High
`Course`	Inner Value Object	Part of `Menu proposal` payload with `1..10` cardinality[cite: 1].	Med
`Unit`	Enum Value Object	Closed unit list (`Piece`, `Cup`, `Table Spoon`, `Pinch`, etc.)[cite: 1].	
`Meal`	Enum Value Object	Closed set (`breakfast`, `lunch`, `dinner`, `supper`)[cite: 1].	High
`UserId`	External ID V0	External user identity[cite: 1, 2].	High
`RecipeId`	External ID V0	Reference to external Recipe bounded context[cite: 1, 2].	High
`HowToStepId`	External ID V0	Reference to step within external Recipe context[cite: 1, 2].	High
`IngredientId`	External ID V0	Reference to external ingredient catalog[cite: 2].	High

2. Aggregates & Invariants Enforcement

```

### Aggregate 1: `HelpRequest`
- **Root**: `HelpRequest`
- **Invariants**:

```

1. `recipe` is mandatory when `type` is `PREPARATION\_STEP\_EXPLANATION` or `INGREDIENT\_SUBSTITUTE` [cite: 2].
2. `howToStep` is optional, but allowed ONLY when `type` is `PREPARATION\_STEP\_EXPLANATION` [cite: 2].
3. `ingredients` list is allowed ONLY when `type` is `INGREDIENT\_SUBSTITUTE` [cite: 2].
4. `preferredProviders` cardinality is `1..2` [cite: 2].
5. `CHEF` provider is exclusive (size = 1) and allowed ONLY for `MENU\_PROPOSAL` [cite: 2].
6. Transition to `CLOSED` status requires at least one `Help` response to be accepted (`hasAcceptedHelp`)

Aggregate 2: `Help`

- **Root**: `Help`
- **Invariants**:
 1. Points to external aggregates by ID only (`HelpRequestId`, `UserId`) [cite: 1].
 2. The `Answer` payload `requestType` must match the target `HelpRequest`'s type [cite: 1].
 3. `MenuProposalAnswer` course count is enforced between `1` and `10` [cite: 1].

3. Open Questions for Domain Expert

1. **Course Dish Value Invariant**: The glossary marks an invariant note on `dish` (`2`) under `course` st
2. **Help Request Re-answering**: The glossary notes "Answered help requests can be still answered" [cite: 1].

Two aggregate roots (HelpRequest and Help) were produced along with 2 domain repositories and a complete decision record. 2 open questions are flagged for domain expert clarification.
